OneCompliance

“Home” View

Specification document

Date: 11-Oct-2021

Version: 1.1

Introduction

This document provides the specification for implementing the Homepage view of OneCompliance application.

Goal

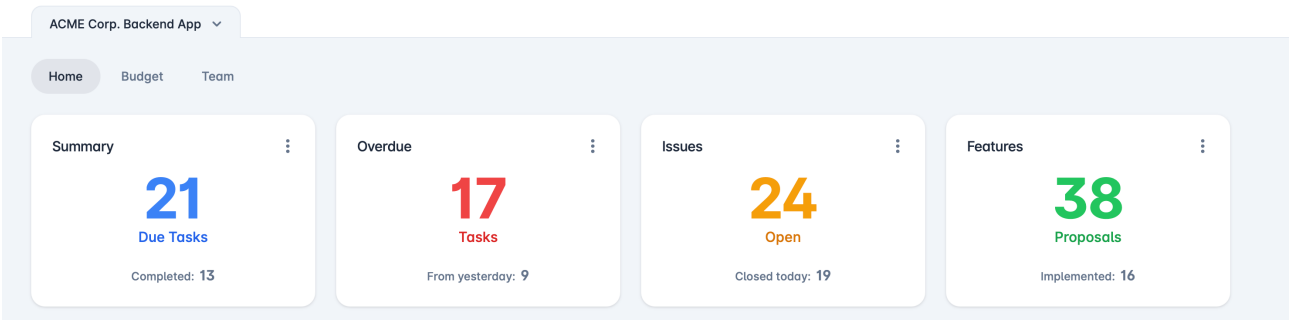
The Homepage must recap the pending and completed actions of the user. The goal is to provide a view easily understandable to provide immediate insights about the tasks status.

Background

The starting point is the fuse angular theme main page and its tiles:

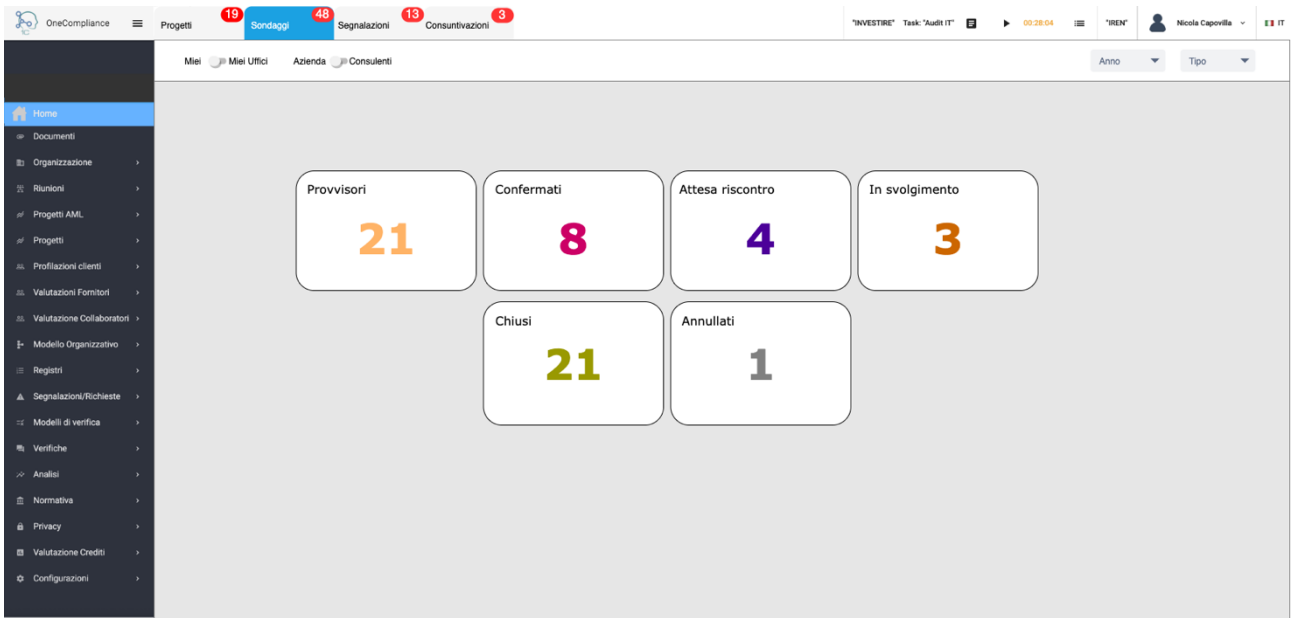
http://angular-material.fusetheme.com/dashboards/project

as shown below. We should use the theme css and visual as much as we can



Mock-up

The Homepage mockup is represented below, visual as said must be better aligned with the theme:



Technical Notes

Homepage

To handle the page, a new type of page must be handled, beside current table and form views.
It must contain an array of tabs (top bar) each one containing

- a description
- optionally a badge with its associated query
- a pointer to the dynamo entry (JSON) describing the subpage

Here below is proposed the related json schema (homepage.schema.json):

```
{
    "title": "Home Page description",
    "type": "object",
    "required": [
        "tabs"
    ],
    "additionalProperties": false,
    "properties": {
        "$schema": {
            "type": "string"
        },
        "tabs": {
            "type": "array",
            "description": "M: Array of tabs",
            "items": {
                "type": "object",
                "required": [
                    "title",
                    "entry"
```

```
                ],
                "additionalProperties": false,
                "properties": {
                    "title": {
                        "type": "string",
                        "description": "M: Label shown on the tab"
                    },
                    "entry": {
                        "type":"string",
                        "description": "M: table entry on DynamoDB that provides
description of the bottom part"
                    },
                    "badgeQuery": {
                        "type": "string",
                        "description": "?: Retrieves a number with label 'badge'
presented (if > 0) as a badge on the tab"
                    }
                }
            }
        }
    }
}
```

Toolbar

Let's review the toolbar of the view entry (linked by the "entry" key above), i.e. this part of the mockup

It must be noted that the toolbar of the usual OneCompliance tables is evolving, this is a snapshot form the application in production right now (table "progetti"), providing new toggle filtering elements, that are table dependent:

In other words, we should introduce a certain degree of programmability of the tool bar in the framework. It has been partially done with the second example, now it is time to evolve it and make it general.
The proposal is to introduce a toolbar key ("toolbar_elements") beside the already defined keys in existing and new tables.
In order to avoid disrupting the existing pages and introducing too much redundancy in all table + form view pages, we keep the general elements in the toolbar hardcoded for each page type and let the user define page specific additional elements to be added in the left (from left to right) or right (from right to left) part of the toolbar.
This means that the table specific recently introduced filtering elements, like "toggle" on page "progetti", must be moved out from "search_keys" and converted into elements of "toolbar_elements".
They must also be put on the same line (differently from today).

The home page toolbar will be instead entirely defined by the new “toolbar_elements” key. This is the format proposed, to be added to the views.schema.json file:

```
        "toolbar_elements": {
            "type": "array",
            "description": "?: Set of page specific toolbar elements",
            "items": {
                "type": "object",
                "required": [
                    "viewType",
                    "label",
                    "fieldName"
                ],
                "additionalProperties": false,
                "properties": {
                    "viewType": {
                        "type": "string",
                        "description": "M: type of the element",
                        "enum": [
                            "toggle",
                            "combobox"
                        ]
                    },
                    "label": {
                        "type": "string",
                        "description": "M: element label, if toggle is primary lable on
the left"
                    },
                    "fieldName": {
                        "type": "string",
                        "description": "M: filtering field name, might or not correspond
to a postgres column"
                    },
                    "position": {
                        "type": "string",
                        "description": "?: indicates the position of the element,
default is left. Elements are displaced in the same order as the array",
                        "enum": [
                            "left",
                            "right"
                        ]
                    },
                    "options": {
                        "$ref": "#/definitions/options"
                    },
                    "queryCond": {
                        "type": "string",
                        "description": "?: postgres query condition (after WHERE
clause), only valid for toggle and combobox elements"
                    },
                    "comboQuery": {
                        "type": "string",
```

```
                        "description": "?: combobox query, must return an array of
[{“id”: Number, “name”: String}], can contain both passed $key$ and local £key£"
                    },
                    "toggleOffLabel": {
                        "type": "string",
                        "description": "?: optional secondary label, only applicable to
two conditions toggle elements"
                    },
                    "toggleOffQueryCond": {
                        "type": "string",
                        "description": "?: postgres query condition (after WHERE
clause), only valid for two conditions toggle elements"
                    }
                }
            }
        }
```

It must be noted that combobox “options” field has now been moved at the bottom of the schema file as a definition.
The filtering elements (toggle and comboboxes) must be treated exactly in the same manner as the “search_keys”. Differently from the latter however, they are always visible and must be applied on selection/toggling.

Tiles

The tiles description must be contained into the specific home page selected on the top tab bar and are addressed by the new key “tiles” within the views.schema.json file. The key must only be taken in account if the page is of type “homepage”, i.e. they are ignored by table and form views.
Here below the JSON addition to views.schema.json:

```
        "tiles": {
            "type": "array",
            "description": "?: Set of tiles with their properties. Only considered if
the view is of type 'homepage'",
            "items": {
                "type": "object",
                "required": [
                    "title",
                    "content"
                ],
                "additionalProperties": false,
                "properties": {
                    "title": {
                        "type": "string",
                        "description": "M: title label of the tile"
                    },
                    "content": {
                        "type": "object",
                        "additionalProperties": false,
                        "required": [
                            "contentType",
                            "fieldName"
                        ],
```

```
                        "properties": {
                            "contentType": {
                                "type": "string",
                                "description": "M: type of the content",
                                "enum": [
                                    "value"
                                ]
                            },
                            "fieldName": {
                                "type": "string",
                                "description": "M: tile field name, typically
corresponding to a postgres table column"
                            },
                            "value": {
                                "type": "string",
                                "description": "M: default value of the content"
                            },
                            "query": {
                                "type": "string",
                                "description": "?: query to define the value of the
content, can use '$param' keys and be enriched by local queryCond  + toolbar filters (as
WHERE conditions)"
                            },
                            "queryCond": {
                                "type": "string",
                                "description": "?: postgres local query condition (after
WHERE clause)"
                            }
                        }
                    }
                }
            }
        }
```

The tile value is clickable, redirecting to the table view of the same table. The filtering conditions as (fieldname, value) provided by the toolbar and the tile itself are passed to the table view as parameters, specifying their nature (that is, coming from toolbar or tile). The table view should then pass these (key, value) parameters to the backend, that should retrieve the related queryCond from the view JSON. Such feature is today not implemented for table view, thus it must be developed.